CS5487 Course Project – Group 98

Group ID: 98
Topic: (Default) Handwritten Digit Classification
Group Members: Zichun Cai (60080748), Wuyue Ma (59673599)

1 Introduction

This project tackles ***Handwritten Digit Classification*** on a subset of the ***MNIST*** dataset, implemented in Python with ***scikit-learn***, ***PyTorch***, and the ***XGBoost*** library.

Using **1-NN** as the baseline, we evaluate in-class methods — ***Logistic Regression*** (linear) and ***Kernel SVM*** (non-linear) — and extend to ensemble methods (***XGBoost***, ***Random Forest***) and deep learning (***MLP***, ***CNN***).

Results show the 1-NN baseline achieves 91.60% while **CNN** reaches **98.10%**, significantly outperforming all traditional models.

2 Methodology

2.1 Classifier Selection

We employ seven classifiers spanning four categories, summarized in Table 1.

Sub-category	**Model Name**	**Key Characteristic**
Baseline	1-NN	Non-parametric, distance-based inference
Linear	Logistic Regression	L_2 regularized probabilistic mapping
Non-linear	Kernel SVM	RBF kernel for high-dimensional projection
Ensemble	Random Forest	Bagging-based parallelized decision trees
Ensemble	XGBoost	Iterative bias optimization via gradient boosting
Neural Network	MLP	Dense connectivity for feature hierarchy
Neural Network	CNN	Spatial inductive bias via convolutions

Table 1. Taxonomy of implemented classification models.

2.2 Pre-processing Pipeline

Raw 784-dimensional vectors are reshaped to 28×28 images, refined via **Median Filtering** and **Gaussian Smoothing**, then centered with **Centroid Centering**. The images are flattened, normalized to $[0, 1]$, and reduced with **PCA**.

Step	Operation	Setting	Purpose
1	Reshape	$784 \to 28 \times 28$	Restore spatial structure for filtering
2	Median Filter	3×3	Mitigate pixel-level noise
3	Gaussian Smoothing	$\sigma = 1.0$	Reduce high-frequency noise
4	Centroid Centering	Shift to center	Standardize digit positioning
5	Flatten	$28 \times 28 \to 784$	Revert to vector format
6	Min-Max Normalization	Scale to $[0, 1]$	Ensure uniform feature contribution
7	PCA	Retain 95% variance	Dimensionality reduction

Table 2. Data preprocessing pipeline.

Model-Specific Adaptations:
1-NN bypasses all preprocessing per course requirements. **XGBoost**, **Random Forest**, and **CNN** skip PCA, as these models handle high-dimensional input natively.

3 Evaluation

3.1 Model Accuracy and Variance Analysis

Each model is evaluated on two complementary trials (Trial 1: designated split; Trial 2: roles reversed). Table 3 reports the results.

Models	Trial 1	Trial 2	Mean Acc.	Std.	F1-Score	Recall
1-NN (Baseline)	0.9135	0.9185	**0.9160**	0.0025	0.9155	0.9160
Logistic Regression	0.8890	0.8775	0.8832	0.0058	0.8827	0.8832
Kernel SVM	0.9490	0.9470	**0.9480**	0.0010	0.9479	0.9480
XGBoost	0.9200	0.9140	0.9170	0.0030	0.9168	0.9170
Random Forest	0.9220	0.9155	0.9187	0.0033	0.9186	0.9187
MLP	0.9425	0.9080	0.9253	0.0172	0.9251	0.9252
CNN	0.9830	0.9790	**0.9810**	0.0020	0.9810	0.9810

Table 3. Experimental results of each algorithm.

Analysis: **Kernel SVM** (94.80%) leads traditional models with the lowest variance (0.0010), which is expected: the RBF kernel maps features into an infinite-dimensional space where digit classes become more separable, while the soft-margin objective ($C = 10$) regularizes against overfitting. LR (88.32%) falls below the 1-NN baseline because multinomial softmax can only learn linear decision boundaries in PCA space — insufficient for the complex topology of handwritten digits. The two ensemble methods, XGBoost (91.70%) and RF (91.87%), only marginally exceed 1-NN; this is likely because the PCA-reduced (or raw) feature space does not strongly favor tree-based splits over nearest-neighbor lookup at this scale. MLP (92.53%) achieves moderate accuracy but exhibits the highest variance (0.0172), reflecting the sensitivity of gradient-based optimization to the random weight initialization and the particular train/test partition. **CNN** (98.10%) is the clear winner: its convolutional layers exploit the 2D spatial structure of digit images — translation invariance and local feature hierarchies — giving it a fundamental advantage that flat-vector methods lack. On the base set, F1-Score and Recall closely track Accuracy for all models, indicating that the per-class performance is well-balanced and no single digit is systematically neglected.

3.2 Challenge Dataset Performance

We further test on the "Challenge" subset (Table 4).

Models	Trial 1	Trial 2	Mean Acc.	Std.	F1-Score	Recall
1-NN (Baseline)	0.6600	0.7067	**0.6833**	0.0233	0.6757	0.6833
Logistic Regression	0.7533	0.7867	0.7700	0.0167	0.7644	0.7700
Kernel SVM	0.8467	0.8667	**0.8567**	0.0100	0.8560	0.8567
XGBoost	0.7667	0.8267	0.7967	0.0300	0.7960	0.7967
Random Forest	0.7733	0.8133	0.7933	0.0200	0.7892	0.7933
MLP	0.7867	0.8067	0.7967	0.0100	0.7946	0.7967
CNN	0.9267	0.9200	**0.9233**	0.0033	0.9195	0.9233

Table 4. Experimental results on the Challenge dataset.

Analysis: All models degrade on the challenge set due to distribution shift — the challenge digits likely come from different writers with different stroke styles, violating the i.i.d. assumption. 1-NN drops most severely (91.60% → 68.33%) because nearest-neighbor classification relies on the training manifold being locally smooth; when test points lie in unfamiliar regions of pixel space, the nearest neighbor becomes meaningless. LR (77.00%) now surpasses 1-NN: even a sub-optimal learned boundary captures *structural* patterns (stroke angles, curvature) that transfer across distributions, whereas raw ℓ_2 distance does not. Kernel SVM holds at **85.67%** thanks to the RBF kernel's ability to define flexible, data-adaptive boundaries. **CNN** remains best at **92.33%**, losing only 6 points — its hierarchical feature extraction (edges → strokes → parts) learns representations that are inherently more invariant to style variations. F1-Score now diverges from Accuracy on weaker models (e.g., 1-NN: F1 = 0.6757 vs Acc = 0.6833), confirming that the distribution shift affects certain digit classes (those with more style variability) disproportionately.

3.3 Error Pattern Analysis

To understand which digits are hardest, we examine per-class recall (Figure 1) and top misclassification pairs.

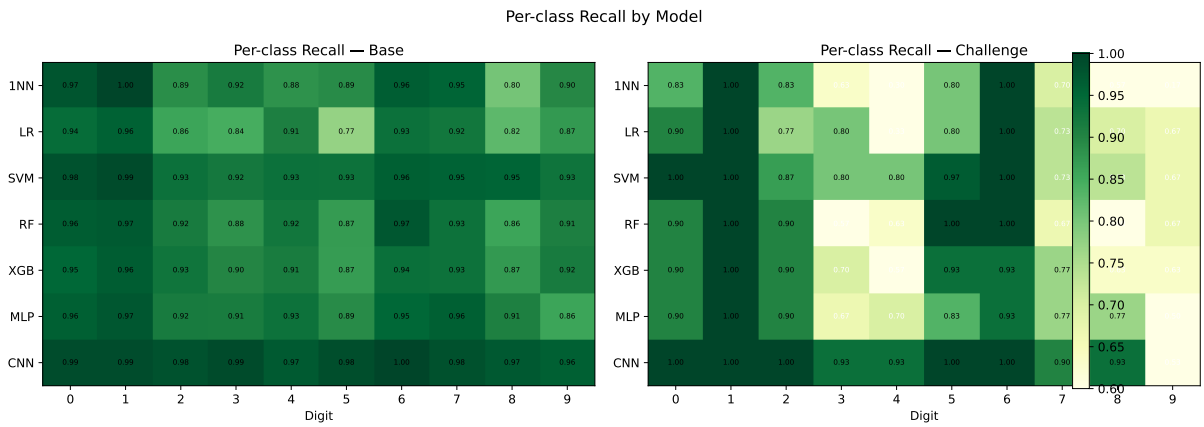


Figure 1. Per-class recall heatmap for all models on base (left) and challenge (right) datasets.

Analysis: On the base set, the dominant error is **4→9** (34 times for 1-NN). This is expected: handwritten 4 and 9 share a closed upper loop, making them difficult to distinguish by pixel-level

or even low-level features. The cluster **3, 5, 8** also shows mutual confusion due to overlapping curved strokes — all three digits consist primarily of arcs and loops that produce similar feature vectors after PCA projection. LR struggles especially with **3↔5** because a linear hyperplane in PCA space cannot capture the subtle curvature differences that distinguish these two digits. On the challenge set, **9→1** and **4→1** emerge as dominant errors for 1-NN, suggesting that the challenge data contains thinner, more elongated strokes: when a 4 or 9 is written with a single vertical stroke, its pixel representation becomes indistinguishable from 1 under ℓ_2 distance. CNN's remaining errors concentrate on the inherently ambiguous **4↔9** and **9→7** pairs — even with learned hierarchical features, these digit pairs overlap near the decision boundary, representing the practical limit of visual discriminability on this dataset.

3.4 Hyperparameter Sensitivity Analysis

Exhaustive grid searches are impractical with our from-scratch code. We first quantify the speed gap, then use *sklearn* for the sweeps.

3.4.1 From-Scratch vs Library Speed Comparison

All benchmarks are performed under the environment listed in Table 5.

Component	**Specification**
CPU	Intel Core i9-14900 (24 cores)
RAM	64 GB DDR5
GPU	NVIDIA RTX A1000 X2 (used only for CNN)
OS	Ubuntu 24.04 LTS
Python	3.12

Table 5. Hardware and software environment.

Table 6 compares training time on Trial 1 under identical preprocessing.

Component	**Implementation**	**Acc.**	**Time (s)**	**Ratio**
PCA	From-scratch (`eigh`)	—	0.07	0.2× (faster)
	sklearn PCA	—	0.33	
Logistic Regression	From-scratch (L-BFGS-B)	0.8810	0.16	9× slower
	sklearn LR	0.8805	0.02	
Kernel SVM	From-scratch (SMO, OvR)	0.9490	15.1	145× slower
	sklearn SVC	0.9415	0.10	
Random Forest	From-scratch	0.9155	16.3	86× slower
	sklearn RF	0.9133	0.19	

Table 6. From-scratch vs *sklearn* training speed on Trial 1.

Analysis: Our from-scratch PCA is actually faster (0.07 s vs 0.33 s) because it directly calls `numpy.linalg.eigh` on the covariance matrix, whereas sklearn's `PCA` includes additional overhead for input validation, automatic solver selection, and numerical stability checks. For classifiers, the gap reverses dramatically: LR is 9× slower, RF 86×, and SVM 145× slower. The SVM gap is the largest because our simplified SMO algorithm performs $O(n^2)$ kernel evaluations per pass in interpreted Python, whereas sklearn's `libsvm` backend is written in optimized

C with efficient kernel caching and shrinking heuristics. Similarly, our RF builds trees with Python-level loops over feature splits, while sklearn uses parallelized Cython code. Despite the speed disadvantage, our from-scratch SVM achieves *higher* accuracy (94.90% vs 94.15%), likely because the SMO convergence criteria differ — our implementation runs more passes, effectively performing more thorough optimization. Given that a single SVM grid search over 35 (C, γ) combinations would require $35 \times 15\text{s} \approx 9$ minutes with our code versus $\sim$4 s with sklearn, all subsequent hyperparameter sweeps use sklearn.

3.4.2 SVM Kernel Comparison

The kernel determines the feature space for SVM's separating hyperplane. We sweep $C \in \{10^{-3}, \ldots, 10^{3}\}$ across five kernels (Linear, RBF, Poly, Sigmoid).

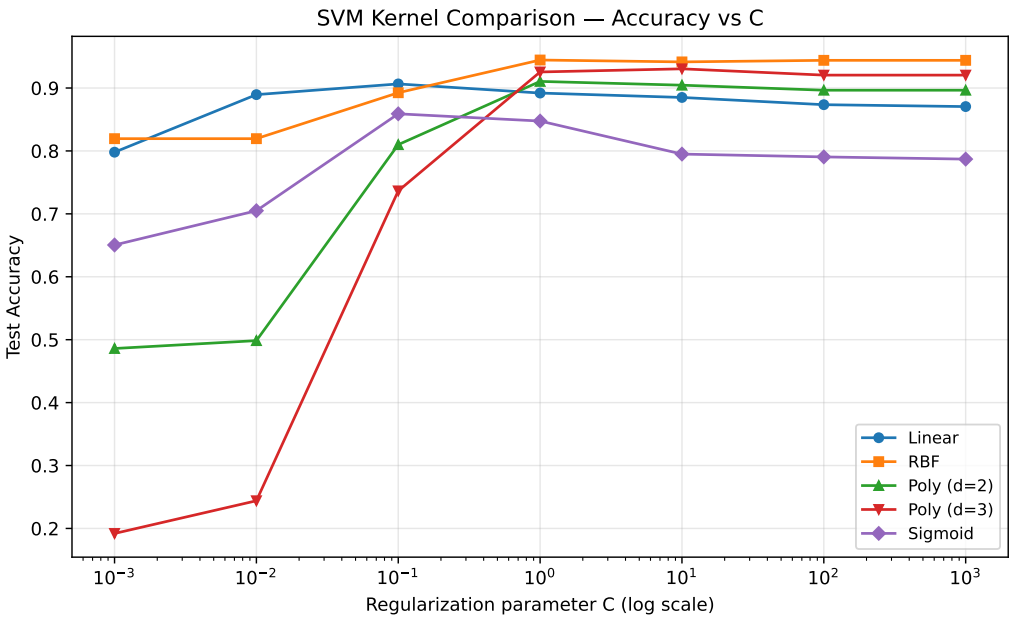


Figure 2. SVM kernel comparison — test accuracy as a function of C on Trial 1.

Analysis: The **RBF kernel** dominates for $C \geq 1$ ($\approx$94.4%), which is theoretically expected: RBF maps input features into an infinite-dimensional Hilbert space via $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$, enabling the SVM to construct highly non-linear decision boundaries. The Linear kernel peaks at $C = 0.1$ (90.7%) then degrades with larger C — increasing C forces the linear model to fit training noise, confirming that the 10-class digit boundaries in 32-dimensional PCA space are intrinsically non-linear. Polynomial kernels ($d = 2, 3$) approach RBF at high C but exhibit less stable convergence. Sigmoid performs poorly throughout, consistent with its known theoretical issues (it does not always yield a valid positive semi-definite kernel matrix).

3.4.3 SVM Grid Search Visualization

For RBF, C controls the bias-variance trade-off and γ controls kernel width. We search (C, γ) with 5-fold CV.

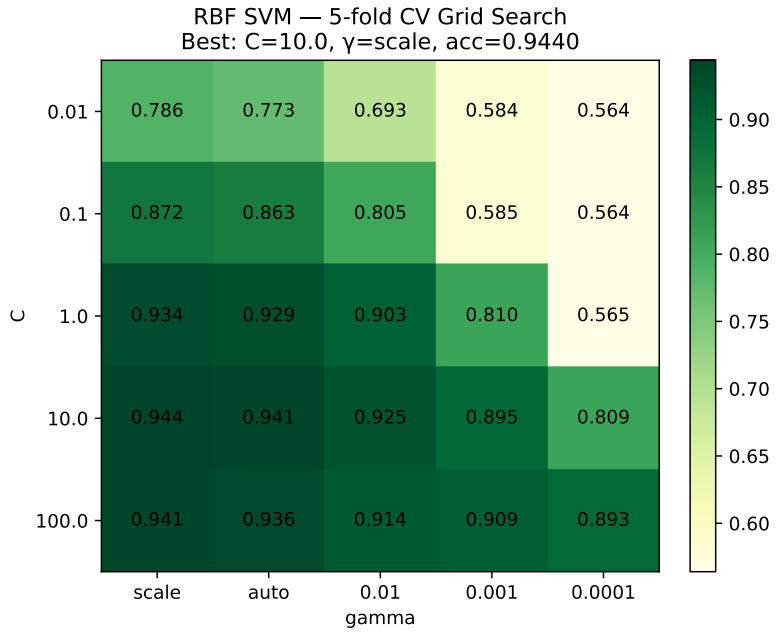


Figure 3. 5-fold CV accuracy heatmap over (C, γ) for RBF Kernel SVM on Trial 1.

Analysis: Best configuration: $C = 10$, $\gamma =$ `scale` (94.40%). The `scale` setting computes $\gamma = 1/(d \cdot \mathrm{Var}(X))$, which automatically adapts to the feature dimensionality and spread — this outperforms all hand-tuned γ values, demonstrating that data-driven parameter calibration is more robust than manual selection. Accuracy saturates for $C \geq 10$: beyond this point, the margin is already narrow enough to correctly classify most training samples, and further increasing C only risks overfitting to outliers without improving generalization.

3.4.4 Effect of PCA Variance Ratio

PCA projects onto top eigenvectors. Too few components lose signal; too many preserve noise. We sweep 0.70–0.99 for LR and SVM.

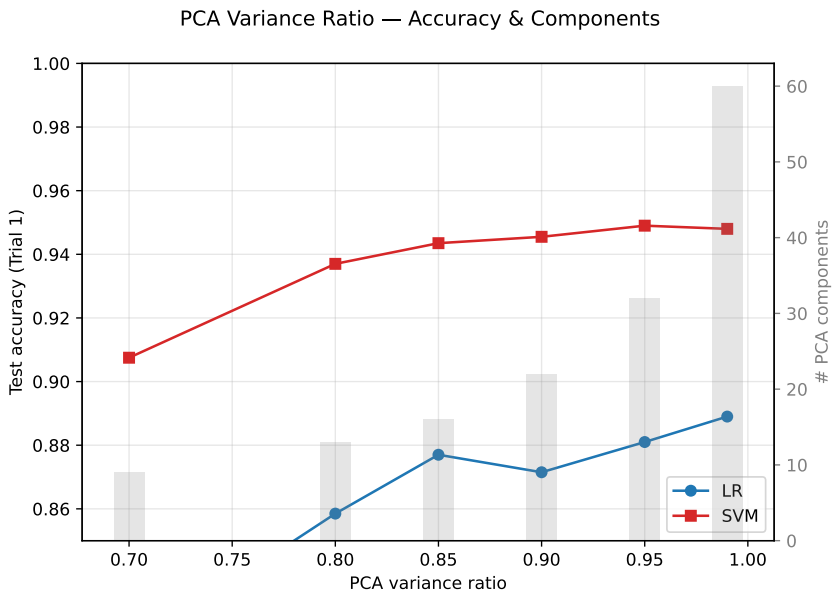


Figure 4. Effect of PCA variance ratio on test accuracy and number of retained components (Trial 1).

Analysis: SVM peaks at **0.95** (≈32 dimensions). Beyond this, the additional principal components capture low-variance directions that are dominated by pixel noise; in the RBF kernel's distance computation, these noisy dimensions inflate $\|\mathbf{x} - \mathbf{x}'\|^2$ and degrade the kernel's discriminative power. LR, by contrast, improves monotonically to 0.99 (≈60 dimensions): a linear classifier benefits from any feature with non-zero signal-to-noise ratio, since its $\mathbf{w}^\top \mathbf{x}$ projection can learn to down-weight noisy components via regularization. This opposite behavior motivates per-model PCA settings: 0.95 for SVM, 0.99 for LR.

3.4.5 Linear PCA vs Kernel PCA

Kernel PCA applies PCA in an implicit high-dimensional space. We compare Linear PCA against seven Kernel PCA variants across 10–160 dimensions.

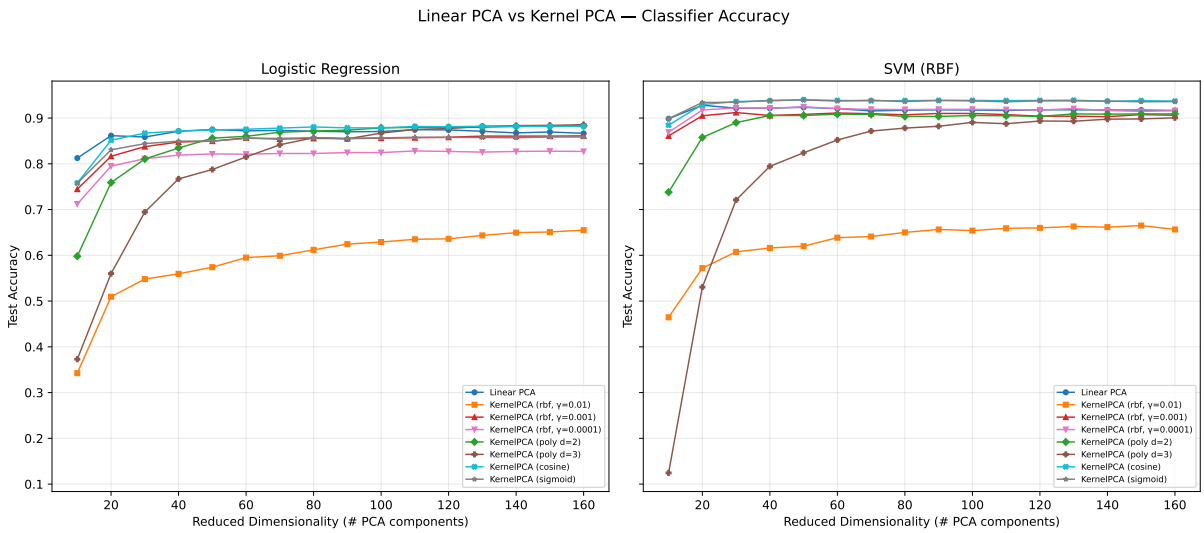


Figure 5. Linear PCA vs Kernel PCA — test accuracy across reduced dimensionalities on Trial 1.

Analysis: **Linear PCA** matches or outperforms all Kernel PCA variants for both classifiers. This indicates that the dominant variance structure of handwritten digit images lies along linear directions in pixel space — the digit manifold, while high-dimensional, is well-approximated by a linear subspace. Kernel PCA with small RBF γ (0.01) consistently underperforms because an overly narrow kernel makes distant points appear equally dissimilar, collapsing the eigenspectrum. Given that Linear PCA is also computationally cheaper (no $N \times N$ kernel matrix), there is no justification for using Kernel PCA on this dataset.

3.4.6 Summary

The hyperparameter experiments reveal several consistent patterns. First, the **RBF kernel** is clearly the optimal choice for SVM on this task — its infinite-dimensional feature mapping provides the flexibility needed for 10-class digit separation, while Linear and Sigmoid kernels are fundamentally limited by their restricted hypothesis spaces. Second, the optimal regularization point ($C = 10$, $\gamma = \texttt{scale}$) represents a balance where the model is expressive enough to capture class boundaries without overfitting; the data-adaptive `scale` setting for γ proves more reliable than any hand-tuned value. Third, PCA dimensionality has **opposite effects** on SVM and LR: SVM benefits from aggressive reduction (0.95, ∼32 dims) because the RBF kernel is sensitive to noisy dimensions, whereas LR benefits from retaining more features (0.99, ∼60 dims) because its linear projection can learn to ignore noise via L_2 regularization. Fourth, **Kernel PCA provides no advantage** over standard Linear PCA, confirming that the digit manifold is well-approximated by a linear subspace. Finally, the **speed comparison** (9–145× gap) demonstrates

the practical necessity of using optimized library implementations for systematic hyperparameter exploration, while validating that our from-scratch implementations achieve comparable or even superior accuracy.

4 Conclusion

In this project, we implemented and evaluated seven classifiers for handwritten digit classification on a 4000-sample MNIST subset. The experimental results, supported by systematic hyperparameter analysis, lead to the following conclusions:

1. **CNN achieves the best accuracy** (98.10% on base, 92.33% on challenge) and demonstrates the strongest robustness under distribution shift. Its convolutional architecture exploits spatial locality and translation invariance — properties inherent to digit images — enabling it to learn hierarchical features (edges $\rightarrow$ strokes $\rightarrow$ parts) that generalize across writing styles. This confirms that *inductive bias aligned with data structure* is the single most important factor for classification performance.

2. **Kernel SVM is the strongest traditional model** (94.80%) with the lowest variance (0.0010) across trials. The RBF kernel's ability to construct non-linear decision boundaries in a high-dimensional feature space, combined with the max-margin objective's inherent regularization, makes it both accurate and stable. Among non-deep methods, this represents the upper bound on what can be achieved with a well-tuned classifier.

3. **Preprocessing design has a significant and model-dependent impact.** The image-domain pipeline (median filtering, Gaussian smoothing, centroid centering) provides consistent improvements for all vector-based models by reducing noise and standardizing digit position. However, PCA dimensionality reduction helps SVM and LR but *hurts* CNN, XGBoost, and Random Forest — models that can internally handle or even benefit from the full 784-dimensional feature space.

4. **Linear PCA is sufficient** for this dataset. Seven Kernel PCA variants (RBF, Polynomial, Cosine, Sigmoid) were tested across a range of dimensionalities, and none outperformed standard Linear PCA. This indicates that the digit feature manifold is well-approximated by a linear subspace, and non-linear dimensionality reduction adds computational cost without accuracy gain.

5. **Per-model hyperparameter tuning is essential.** SVM and LR exhibit opposite PCA preferences (0.95 vs 0.99); the choice of kernel type drastically affects SVM performance (RBF: 94.4% vs Sigmoid: <80%); and the (C, γ) grid search reveals a sharp optimum at $C = 10$, $\gamma = \texttt{scale}$. These results underscore that there is no single "default" configuration — each model requires dedicated tuning.

6. **From-scratch implementations are pedagogically valuable but computationally expensive.** Our NumPy-based SVM is 145$\times$ slower than sklearn's C backend, making it impractical for exhaustive hyperparameter search. Nevertheless, implementing these algorithms from scratch deepened our understanding of the underlying optimization procedures (SMO for SVM, L-BFGS for LR, eigendecomposition for PCA) and their convergence behaviors.

Overall, this project demonstrates that the path from a 91.60% baseline (1-NN) to 98.10% (CNN) involves not just model selection, but a careful interplay of preprocessing, dimensionality reduction, kernel choice, and regularization. Even within traditional methods, thoughtful tuning (Kernel SVM at 94.80%) can close much of the gap.

Appendix: Implementation Details

Components are categorized as **From-Scratch (S)** (NumPy/SciPy) or **Library-Based (L)** (scikit-learn, PyTorch, XGBoost). All in-class models and core preprocessing are from scratch.

Table 7. Component implementation and dependencies

Component	Source (Lib)	Technical Notes
Pre-processing	From-Scratch (`np`)	Median/Gaussian filters, sub-pixel shift, PCA
1-NN	From-Scratch (`np`)	Vectorized ℓ_2 distance implementation
Logistic Regression	From-Scratch (`np, sp`)	Multinomial softmax + L_2; L-BFGS-B optimizer
Kernel SVM	From-Scratch (`np`)	Simplified SMO algorithm; One-vs-Rest; RBF
Random Forest	*scikit-learn* (L)	500-tree `RandomForestClassifier`
MLP	*scikit-learn* (L)	`MLPClassifier`; Arch: (512, 256, 128)
CNN	*PyTorch* (L)	LeNet-style: 2×Conv-BN-ReLU + Dropout
XGBoost	*XGBoost* (L)	Histogram-based tree; 150 rounds; $\eta = 0.1$

Table 8 summarizes the final hyperparameters, determined through the analyses in Section 3.3.

Table 8. Model configuration summary

Model	Preprocessing	PCA	Key Hyperparameters
1-NN	`noprocess`	—	$k = 1$
Logistic Regression	`wholeprocess`	0.99	$C = 1.0$; `max_iter` = 300
Kernel SVM	`wholeprocess`	0.95	$C = 10$; $\gamma =$ `scale`
Random Forest	`noPCA`	—	500 trees; `max_features` = $\sqrt{D}$
XGBoost	`noPCA`	—	150 rounds; $\eta = 0.1$; depth = 6
MLP	`wholeprocess`	0.95	Arch: (512, 256, 128); `lr` = 0.005
CNN	`noPCA`	—	50 epochs; `lr` = 0.001; batch = 64

Environment dependencies (Table 9):

Table 9. Project environment dependencies (`requirements.txt`)

Package	Category	Primary Role
`numpy`	Numerical	Linear algebra for scratch models and preprocessing
`scipy`	Optimization	Scientific primitives (L-BFGS-B) for LR model
`scikit-learn`	ML Library	Ensemble models, MLP, and evaluation metrics
`xgboost`	Gradient Boosting	High-performance gradient boosting framework
`torch`	Deep Learning	CNN definition, GPU acceleration, and training loop
`joblib`	Serialization	Model persistence (Save/Load)
`matplotlib`	Visualization	Result plotting and image export to `results/images/`

Appendix: Quick-Start Guide

The project root is structured as follows (non-essential files omitted):

```
.
 main.py                   # unified entry point
 Makefile                  # make targets (see below)
 requirements.txt          # Python dependencies
 check_env.py              # environment & data verifier
 pipeline/                 # models and preprocessing
    preprocess.py         #   preprocessing pipeline
    OneNN.py              #   1-NN (from scratch)
    LogisticRegression.py#   Softmax LR (from scratch)
    KernelSVM.py          #   RBF SVM (from scratch)
    RandomForest.py       #   sklearn RandomForest
    XGBoost.py            #   XGBoost wrapper
    MLP.py                #   sklearn MLP
    CNN.py                #   PyTorch LeNet-style CNN
 evaluation/               # hyperparameter analysis scripts
 utils/                    # I/O helpers (data, model, results)
 data/raw/                 # MNIST base + challenge data
 data/trained-model/       # saved model checkpoints
 results/                  # outputs (JSON, images, eval figures)
```

All operations are invoked via `make` from the project root:

Command	Description
`make all`	Run the full pipeline end-to-end: build → checkenv → train → base → challenge → eval.
`make build`	Install all Python dependencies from `requirements.txt`.
`make checkenv`	Print the Python version, then run `check_env.py` to verify that all packages and data files are present.
`make train`	Train all models on both trials and save checkpoints to `data/trained-model/`.
`make base`	Evaluate all models on the base test set (loads saved models; trains if missing). Outputs confusion matrices to `results/images/`.
`make eval`	Run the four hyperparameter sensitivity analysis scripts in `evaluation/`. Outputs figures to `results/eval_figures/`.
`make challenge`	Evaluate all models on the challenge dataset.
`make clean`	Remove saved models and Python caches.

Table 10. Available `make` targets.

Recommended workflow:

```
make all          # run everything end-to-end
```

Or step by step:

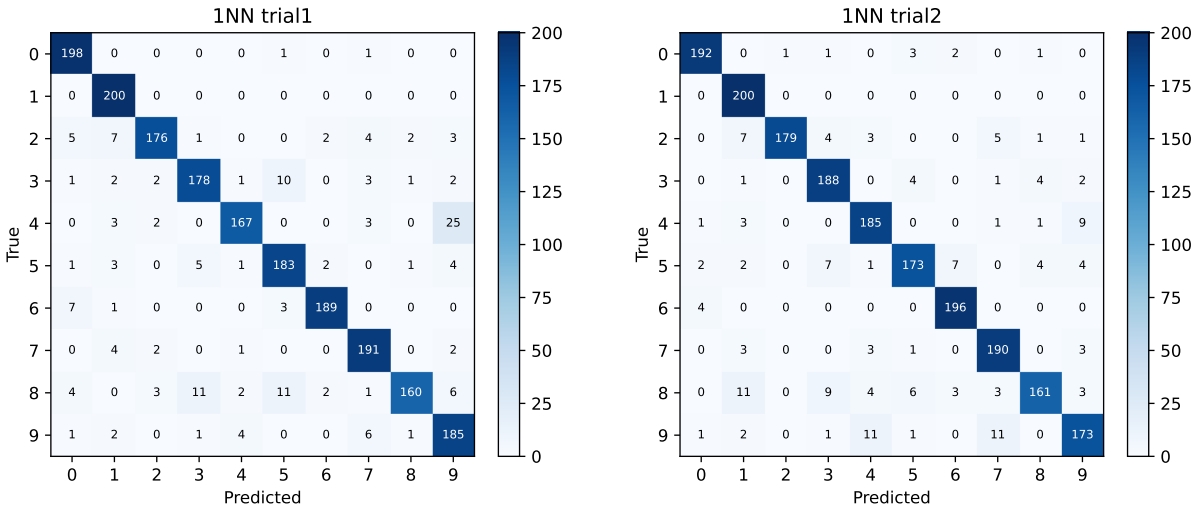
```
make build        # 1. install dependencies
make checkenv     # 2. verify environment
make train        # 3. train all models
```

```
make base         # 4. evaluate on base test set
make challenge    # 5. evaluate on challenge set
make eval         # 6. generate analysis figures
```

Appendix: Confusion Matrices

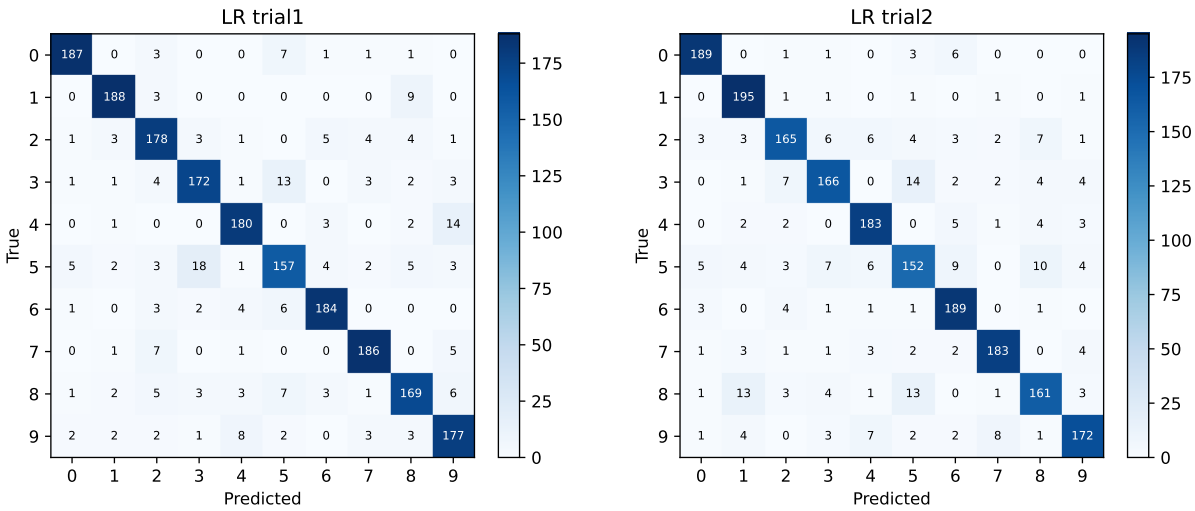
The following figures present per-model confusion matrices for both experimental trials. Each figure shows Trial 1 (left) and Trial 2 (right) side by side.

B.1 Base Test Set



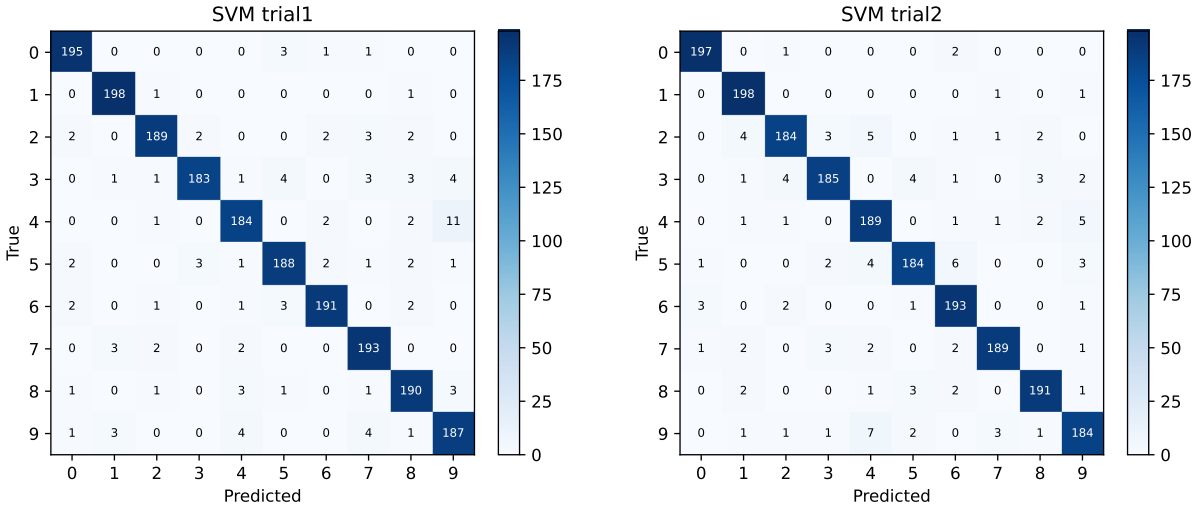
(a) Trial 1 **(b)** Trial 2

Figure 6. 1-NN (Baseline) — Base test set.



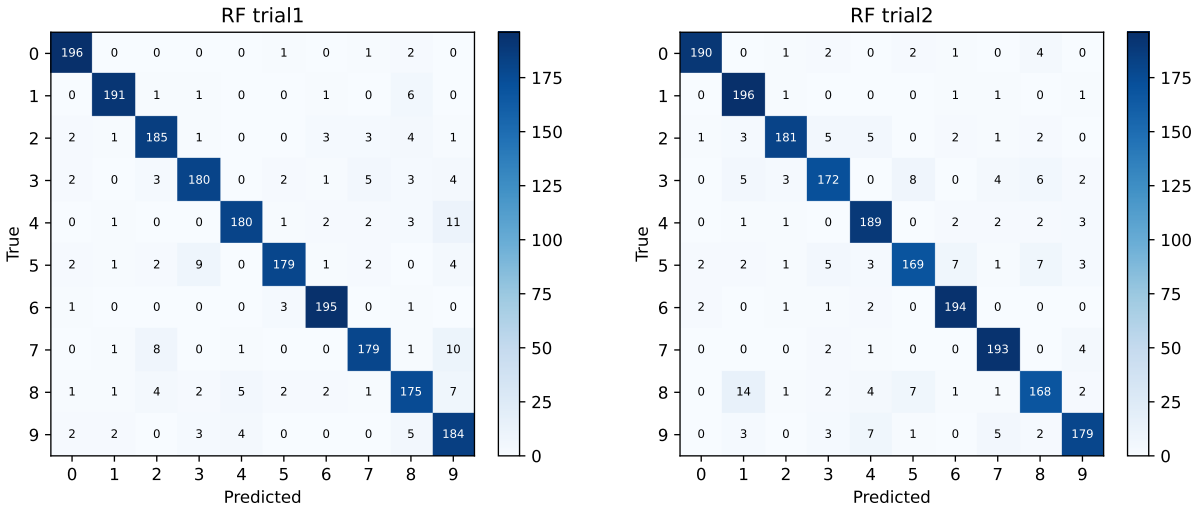
(a) Trial 1 **(b)** Trial 2

Figure 7. Logistic Regression — Base test set.



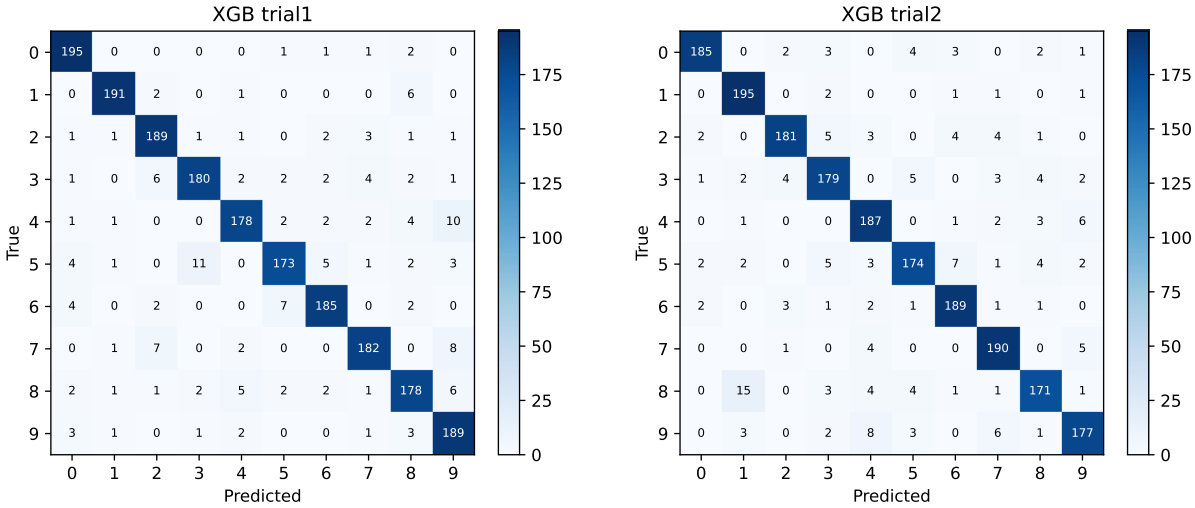
(a) Trial 1 **(b)** Trial 2

Figure 8. Kernel SVM — Base test set.



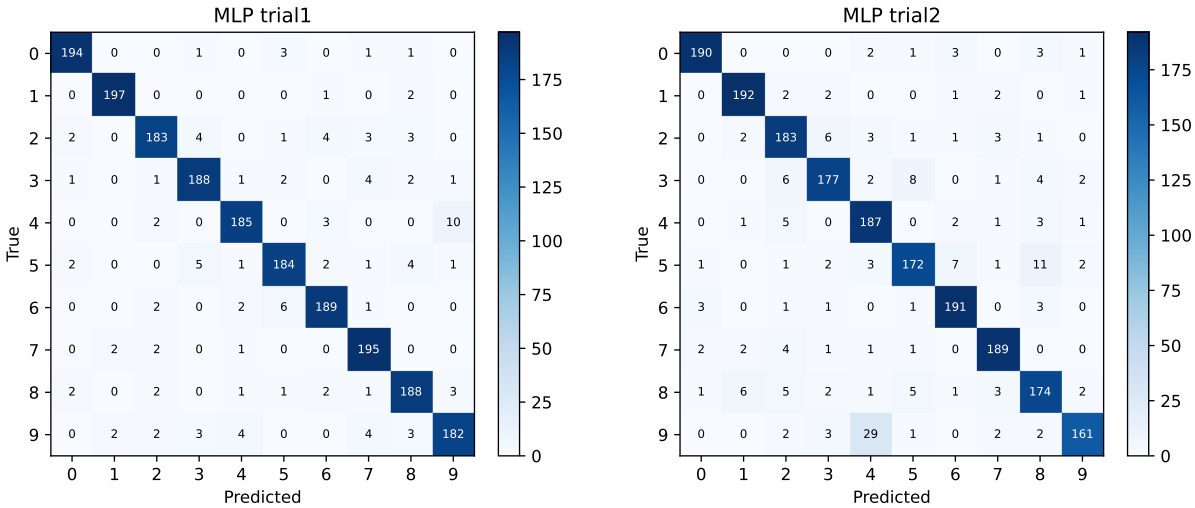
(a) Trial 1 **(b)** Trial 2

Figure 9. Random Forest — Base test set.



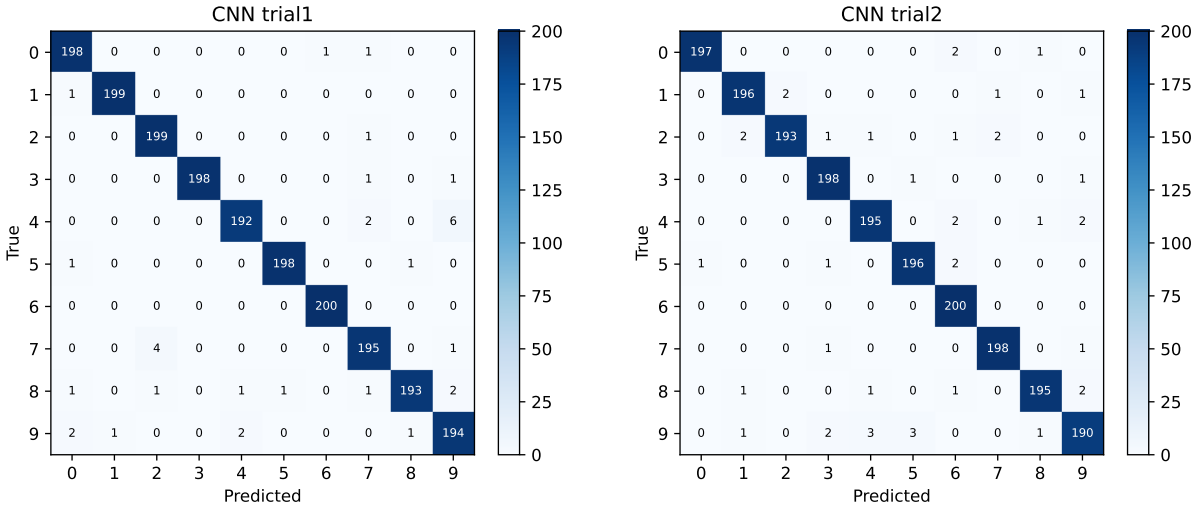
(a) Trial 1 **(b)** Trial 2

Figure 10. XGBoost — Base test set.



(a) Trial 1 **(b)** Trial 2

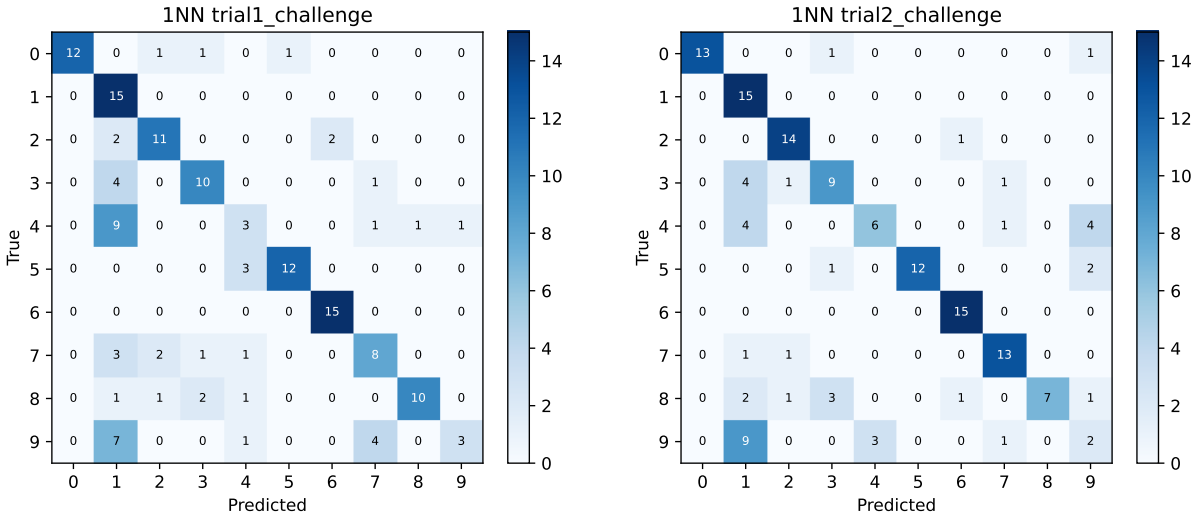
Figure 11. MLP — Base test set.



(a) Trial 1 **(b)** Trial 2

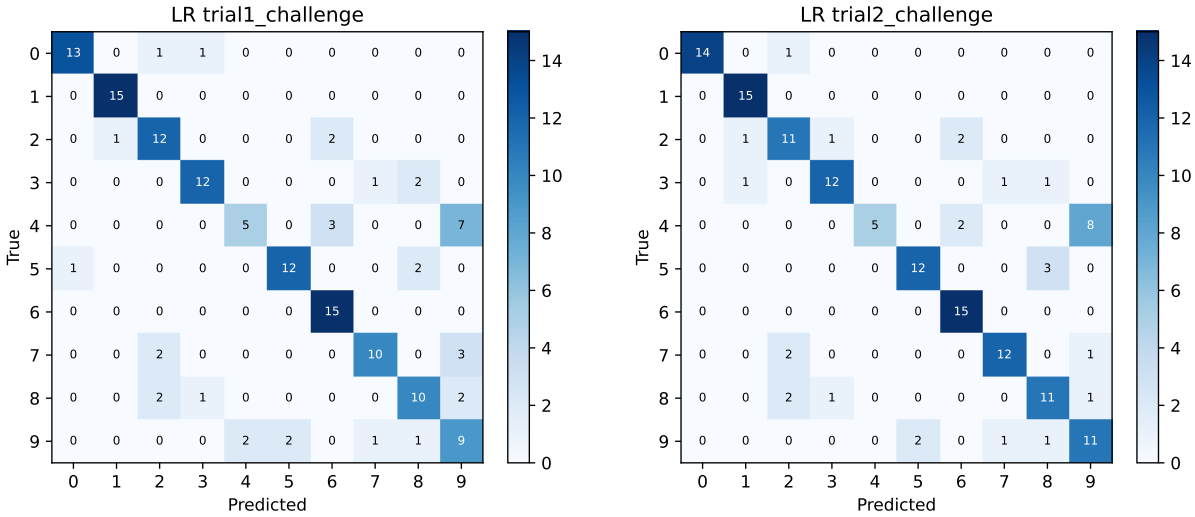
Figure 12. CNN — Base test set.

B.2 Challenge Dataset



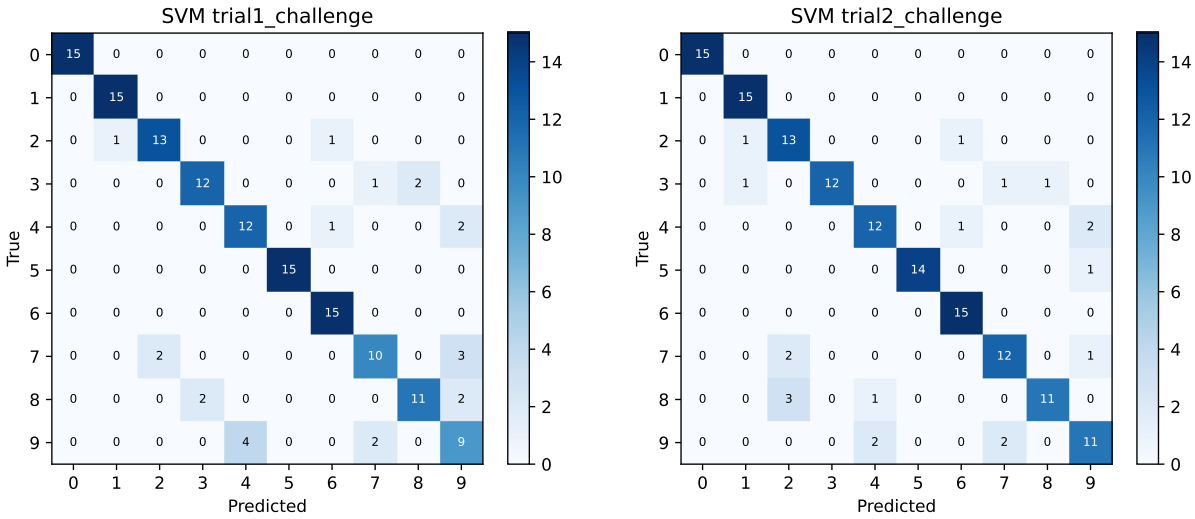
(a) Trial 1 **(b)** Trial 2

Figure 13. 1-NN (Baseline) — Challenge dataset.



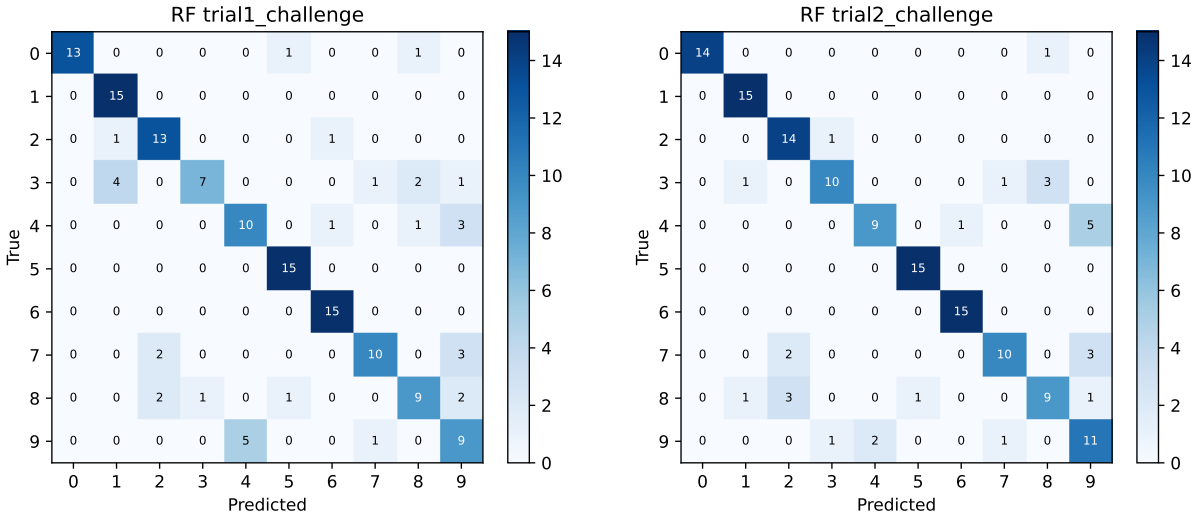
(a) Trial 1 **(b)** Trial 2

Figure 14. Logistic Regression — Challenge dataset.



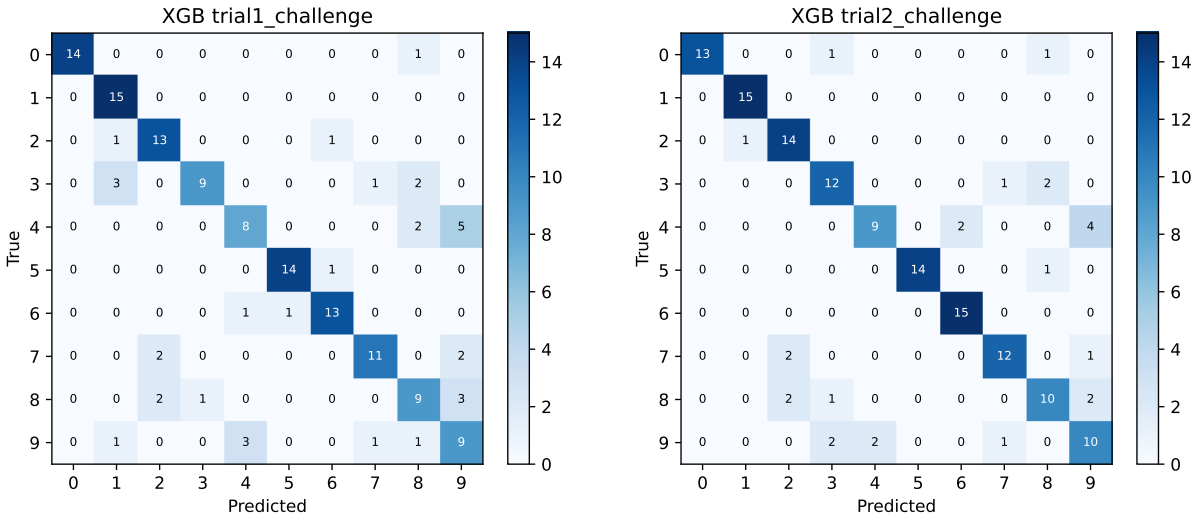
(a) Trial 1 **(b)** Trial 2

Figure 15. Kernel SVM — Challenge dataset.



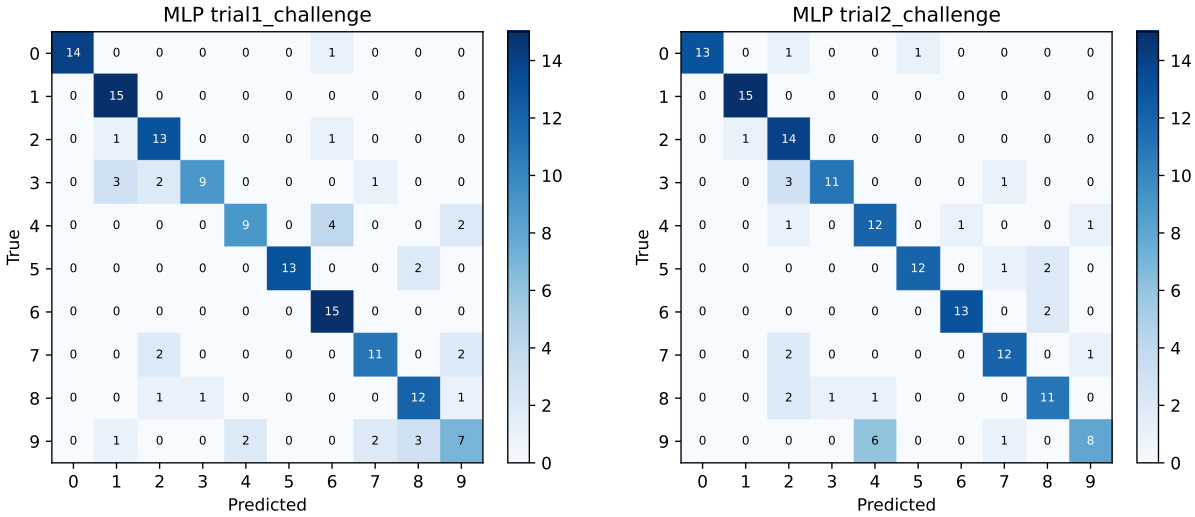
(a) Trial 1 **(b)** Trial 2

Figure 16. Random Forest — Challenge dataset.



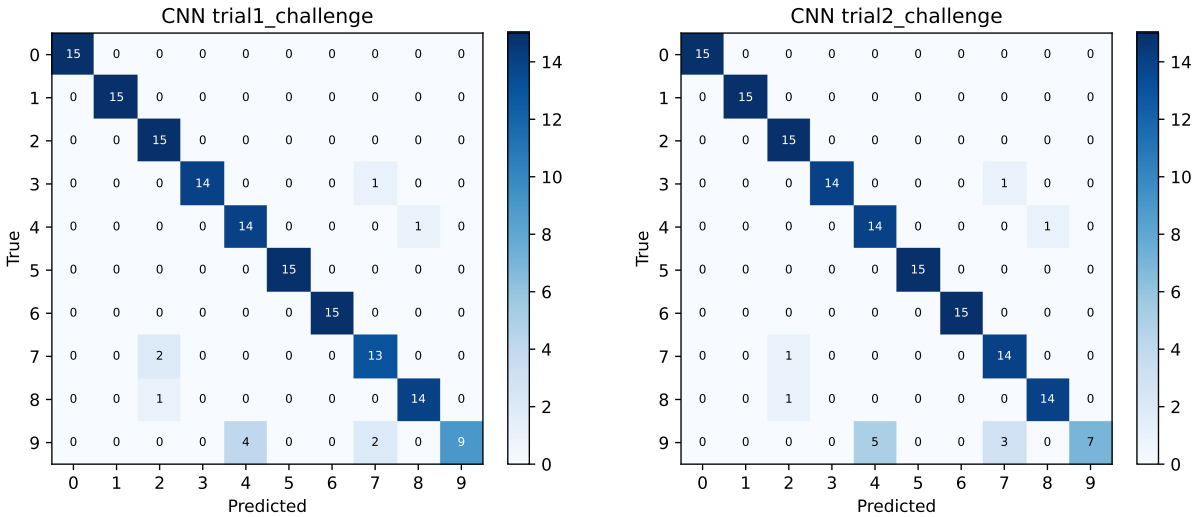
(a) Trial 1 **(b)** Trial 2

Figure 17. XGBoost — Challenge dataset.



(a) Trial 1 **(b)** Trial 2

Figure 18. MLP — Challenge dataset.



(a) Trial 1 **(b)** Trial 2

Figure 19. CNN — Challenge dataset.